Data Analytics Case Study

Sales & Marketing Performance Questionnaire (SQL & Pandas)

Problem Statement

You are a data analyst tasked with evaluating customer data to uncover insights regarding sales, profitability across different states, marketing expenditures, and operational factors like COGS (Cost of Goods Sold) and budget variances.

Your objective is to write functional data-querying operations (SQL scripts or Python Pandas code) to process a sample dataset and answer the questions below.

Dataset Schema Reference

- **FactTable (4,200 rows):** Date, ProductID, Profit, Sales, Margin, COGS, Total_Expenses, Marketing, Inventory, Budget_Profit, Budget_COGS, Budget_Margin, Budget_Sales, Area_Code
- **ProductTable (13 rows):** Product_Type, Product, ProductID, Type
- **LocationTable (156 rows):** Area_Code, State, Market, Market_Size

Phase 1: Basic Aggregations & Filtering

1. Count the number of unique states present in the LocationTable.

2. Filter the ProductTable to find the total count of products where the Type is 'Regular'.

3. Retrieve the total amount spent on marketing for the product with a ProductID of 1.

4. Find the minimum value recorded in the Sales column.

5. Determine the maximum value recorded for Cost of Goods Sold (COGS).

6. Filter and display all columns from the ProductTable where the Product_Type is 'Coffee'.

7. Extract all records from the FactTable where Total_Expenses are strictly greater than 40.

8. Calculate the average Sales for the area code 719.

Phase 2: Joins, Grouping, and Sorting

9. Calculate the total profit generated specifically by the state of 'Colorado' (requires joining FactTable and LocationTable).

10. Calculate the average Inventory grouped by each unique ProductID.

11. Retrieve a list of all states from the LocationTable sorted in ascending alphabetical order.

12. Group the data by product attributes to display the average budget metrics, filtering the final output to only include groups where the average Budget_Margin is greater than 100.

13. Calculate the sum of total Sales recorded on the specific date 2010-01-01.

14. Calculate the average Total_Expenses broken down by each unique combination of ProductID and Date.

15. Perform a multi-table join to generate a single consolidated view containing the following fields: Date, ProductID, Product_Type, Product, Sales, Profit, State, and Area_Code.

Phase 3: Advanced Transformations & Analytics

16. Generate a dense rank (ranking without any gaps in sequential numbers) of records based on their Sales values in descending order.

17. Aggregate the data to find the total Profit and total Sales for each individual State.

18. Compute the total Profit and total Sales broken down by both State and Product name.

19. Create a new calculated column that models a 5% increase in current sales values (New_Sales = Sales * 1.05).

20. Identify the maximum profit earned, along with its corresponding ProductID and Product_Type.

21. Code Implementation:
• SQL Choice: Create a Stored Procedure that takes Product_Type as an input parameter and returns matching rows.
• Pandas Choice: Create a reusable Python function that accepts a DataFrame and a Product_Type string parameter to return the filtered subset.

22. Create a conditional flag or new column named Financial_Status where if Total_Expenses is less than 60, it returns 'Profit', otherwise it returns 'Loss'.

23. Compute the total weekly sales value inclusive of Date and ProductID details, structuring the output hierarchical aggregation (using a Roll-up style rollup/subtotal operation).

24. Perform set operations (specifically Set Union and Set Intersection) leveraging the Area_Code attributes across the dataset tables.

25. Code Implementation: Write a parameterized user-defined function (UDF) or Python function that dynamically filters the product catalog based on user-specified preference criteria.

Phase 4: Data Mutations & String Formatting

26. Simulate a data update: Modify the Product_Type from 'Coffee' to 'Tea' for ProductID 1, and then demonstrate how you would roll back/revert this operational state change.

27. Filter and display the Date, ProductID, and Sales for all entries where Total_Expenses fall between 100 and 200 (inclusive).

28. Write a command to delete/drop all records from the ProductTable where the product Type is 'Regular'.

29. Isolate the 5th character from the string entries in the Product column and return its corresponding numeric ASCII/Unicode value.